

Final Project

Luis Gilbuena, Matthew Merritt, Michael Merritt, Anton Petrenko

May 7, 2025

1 Approach

1.1 Primary Data Structure

The key idea to our approach was to construct a dictionary that would use n -grams (represented as tuples) as keys and return the most likely part of speech for the word given that specific n -gram. This was done by going through the dataset and constructing n -grams from each sentence using the actual part of speech, and tallying the results as we went. Once all of the dataset was parsed and added to the dictionary, the dictionary was then separated into two sets:

- **“Unanimous”** n -grams that always resulted in the same part of speech during training regardless of additional context, such as a bigram that always resulted in the word being tagged as a verb.
- **“Non-unanimous”** n -grams that could result in different parts of speech with additional context, such a trigram that could result in being tagged as a noun or a verb based on the words before it.

Once these two sets are separated, we then store only the most likely option for the part of speech for every n -gram to return when queried, and remove all of the additional tallying information.

1.2 n -gram Lookup Order

Using the two sets mentioned above, we modified our approach to use the smallest amount of information needed to get a definitive answer. Specifically, we first check the unanimous set to see if the unigram always leads to the same part of speech regardless of what comes next, followed by the same check for the bigram, trigram, and quadgram. Assuming that there was no single answer beforehand, we then use the non-unanimous set to find the most likely result from the quadgram, trigram, bigram, or unigram, checking them in that order. If there are still no results for the given n -gram in either set, we default to assuming that the word is a noun (“NN”), the most common part of speech.

1.3 Jackknife Training

For the training process, we used a jackknife technique to split the data into separate training and testing datasets, with each slice created by the jackknife procedure setting aside 1000 sentences for the testing dataset, and using the remaining sentences as the training dataset. Since there were 47959 sentences in total, this means that 48 different training and testing sets were created and tested (the final testing set was only 959 sentences rather than the full 1000, since the dataset did not split evenly). The accuracy was then taken as the average of all 48 different trials.

1.4 Separate Implementations

To test out a few different implementation ideas we had as a group, we created three separate implementations of the aforementioned approach.

- **Luis’s implementation**, which does x
- **Anton’s implementation**, which does y
- **The Merritt’s implementation**, which does z

Each implementation will be discussed more in depth in its own section below.

2 Implementations

This section focuses on the Python implementations of the general approach described above.

2.1 Luis's Implementation

2.1.1 Implementation

Describe how your approach differs from the overall approach (what you did differently in specific, like data structure implementation).

```
1 print('Any example code should be formatted like this')
```

2.1.2 Results

The accuracy of this implementation of the n -gram tagger was 100%, and the program had a total runtime of 100 seconds.

2.2 Anton's Implementation

2.2.1 Implementation

Describe how your approach differs from the overall approach (what you did differently in specific, like data structure implementation).

```
1 print('Any example code should be formatted like this')
```

2.2.2 Results

The accuracy of this implementation of the n -gram tagger was 100%, and the program had a total runtime of 100 seconds.

2.3 The Merritt's Implementation

2.3.1 Implementation

Describe how your approach differs from the overall approach (what you did differently in specific, like data structure implementation).

```
1 print('Any example code should be formatted like this')
```

2.3.2 Results

The accuracy of this implementation of the n -gram tagger was 100%, and the program had a total runtime of 100 seconds.

3 Reflection

Some sort of conclusion goes here.